JIT (Just-In-Time) vs. AOT (Ahead-Of-Time) Compilation

Compilation is the process of converting high-level programming code (e.g., Java, Kotlin) into machine code that a device's processor can understand. Android uses two major compilation techniques:

1. **JIT (Just-In-Time) Compilation –** Used in **Dalvik VM** and partially in ART.
2. **AOT (Ahead-Of-Time) Compilation –** Used in **Android Runtime (ART)** by default.

---

1) JIT (Just-In-Time) Compilation

Introduction:

JIT compiles parts of the code **at runtime** (while the application is running). Instead of compiling the entire application in advance, it **compiles only the necessary code on demand**.

How It Works:

- When an app runs, JIT compiles only the parts of the code that are **frequently used** and stores them in memory.
- The next time the app runs, the already compiled code is used, making execution faster.

Example:

- Imagine a **video player app**.
- When the app launches, only the **UI components** are compiled.
- When the user plays a video, **playback-related code** is compiled.
- If a new feature is used (e.g., downloading a video), that specific code is compiled on the spot.

Pros of JIT:

✅ **Faster initial app installation** (since code is compiled at runtime).
✅ **Uses less storage** (as it doesn't store fully compiled code).
✅ **Optimizes frequently used code dynamically** (adapts to usage patterns).

Cons of JIT:

❌ **Slower app execution** (since code is compiled while running).
❌ **Higher battery consumption** (because CPU has to work more).
❌ **More RAM usage** (as compiled code needs to be stored in memory).

---

2) AOT (Ahead-Of-Time) Compilation

Introduction:

AOT compiles the entire application **at the time of installation**, meaning all the app's code is converted into machine code before execution.

How It Works:

- When an app is installed, ART **compiles all the Java/Kotlin code into native machine code** before the user runs it.
- When the app launches, no further compilation is needed, leading to **faster startup times**.

Example:

- A **gaming app** installed on a phone.
- During installation, all the game's logic, graphics, and UI code is compiled into **native machine code**.
- When the user starts the game, it **runs instantly without additional compilation**.

Pros of AOT:

✅ **Faster app launch** (since the code is already compiled).
✅ **Lower battery consumption** (less CPU usage while running).
✅ **Better overall performance** (optimized execution).

Cons of AOT:

❌ **Slower installation time** (as compilation happens during install).
❌ **Larger storage requirement** (compiled code takes more space).
❌ **Less adaptable to dynamic optimization** (since it doesn't adjust during runtime).

Comparison Table: JIT vs. AOT

Feature	JIT (Just-In-Time)	AOT (Ahead-Of-Time)
Compilation Time	At runtime (while app runs)	During installation
App Launch Speed	Slower (code compiles at startup)	Faster (code already compiled)
Performance	Slower	Faster
Battery Efficiency	Higher consumption (CPU works more)	More efficient
RAM Usage	Higher (stores compiled code temporarily)	Lower
Installation Speed	Faster	Slower
Storage Requirement	Lower	Higher
Code Optimization	Adapts dynamically	Pre-optimized

Conclusion:

- **JIT is better for development environments** (e.g., Android Studio uses JIT for fast testing).
- **AOT is better for end-users**, as it improves app performance, battery life, and startup time.
- **Modern Android (ART) uses a mix of both**: **AOT for initial installation** and **JIT for runtime optimizations**.